13. ROS2 Distributed Communication

1. Concept

Multi-machine communication, i.e. distributed communication, refers to a communication strategy that enables data exchange between different hosts over a network.

ROS2 itself is a distributed communication framework that makes it convenient to realize communication between different devices. The middleware that ROS2 is based on is DDS. When in the same network, distributed communication can be achieved through the DDS domain ID mechanism (ROS_DOMAIN_ID). The general process is: before starting the nodes, you can set the value of the domain ID. If different nodes have the same domain ID, they can freely discover each other and communicate; conversely, if the domain ID values are different, communication is impossible. By default, the domain ID used by all nodes at startup is 0. In other words, as long as they are in the same network, you do not need to do any configuration, and different nodes on different ROS2 devices can achieve distributed communication.

The application scenarios of distributed communication are quite broad, such as unmanned vehicle platooning, drone platooning, remote control, and so on. All of these data exchanges rely on distributed communication.

2. Implementation

2.1. Default Implementation

You only need to put the host and the slave [there can be multiple] in the same network, and distributed communication is already achieved. For example, the host and the slave connect to the same WiFi or the same router.

Setting the network of a virtual machine in Windows to [bridge mode] places it in the same network as the host.

Test:

- Here we assume we have two hosts, A and B. They can be any form of host able to connect to the network, for example: a virtual machine, Raspberry Pi, jetson, x86/ARM host, card mainboard. It only requires the same version of the ros2 environment to be installed.

1. Host A executes:

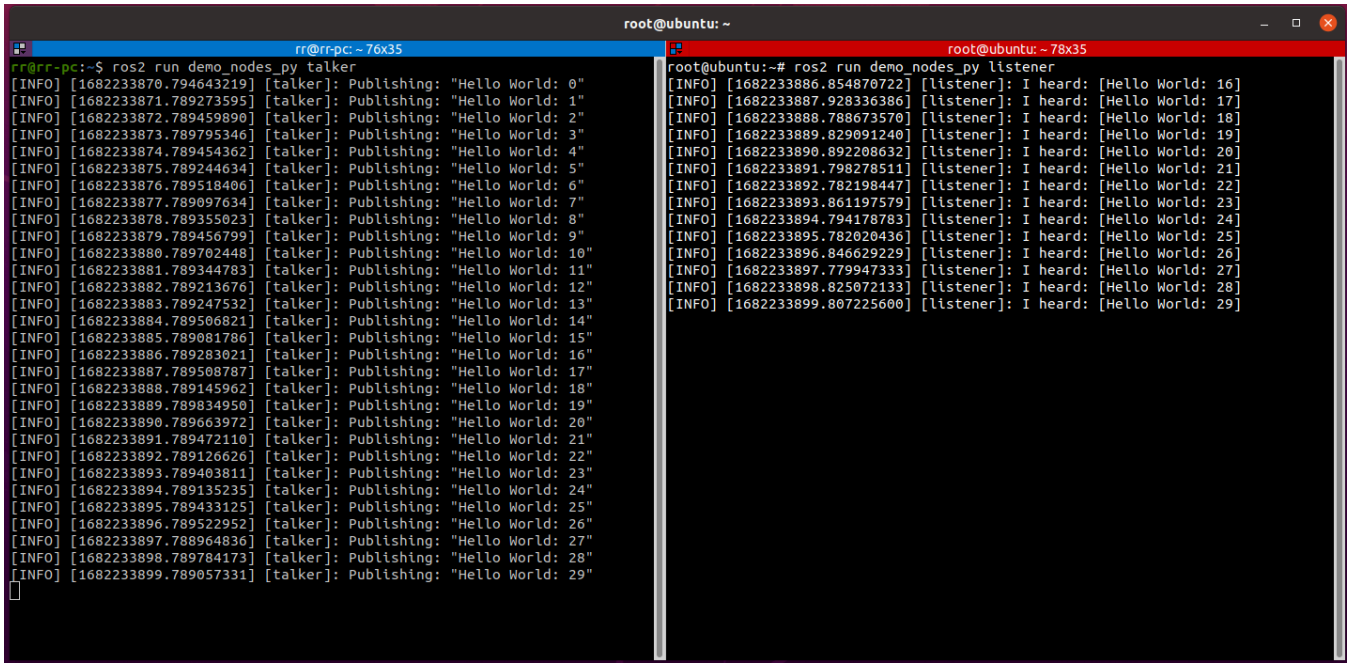
What is demonstrated here is the car running in docker. The network mode used by docker is host mode. Simply put, host mode means sharing the same network with the car, so there is no difference from executing it on the car.

```
ros2 run demo_nodes_py talker
```

2. Host B executes:

```
ros2 run demo_nodes_py listener
```

If the following is displayed: the topic published by the host can be subscribed to in time by the slave, which means multi-machine communication has been achieved



2.2. Distributed Network Grouping

Assume that there are other robots in use in the network you are currently in. In order not to be disturbed by the other robots, you can also set up a group for your robot.

ROS2 provides a DOMAIN mechanism, which is similar to grouping. Only the computers in the same DOMAIN can communicate with each other. We can add the following configuration line to the .bashrc of the host side [car] and the slave side [virtual machine], so that both are assigned to the same group:

```
$ export ROS_DOMAIN_ID=<your_domain_id>
```

If the host side [car] and the slave side [virtual machine] are assigned different IDs, the two cannot communicate with each other, which achieves the purpose of grouping.

2.2.1 Example 1

1. Execute on the host side [car]:

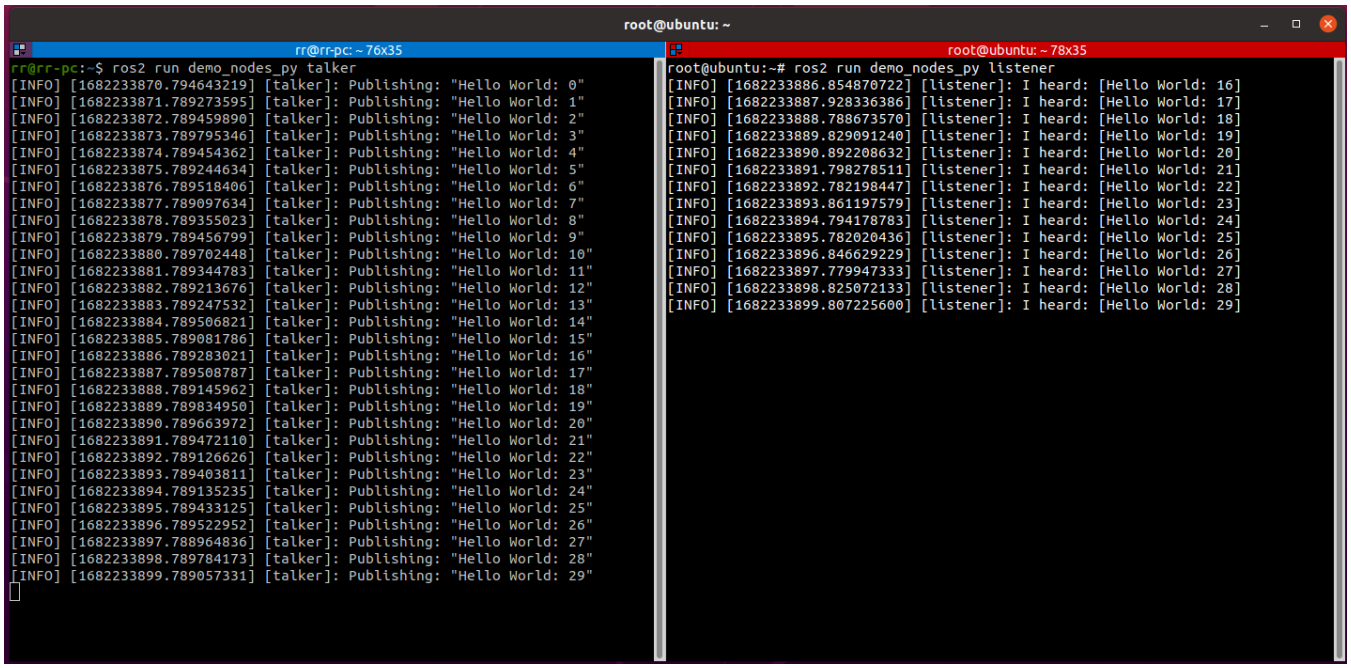
What is demonstrated here is the car running in docker. The network mode used by docker is host mode. Simply put, host mode means sharing the same network with the car, so there is no difference from executing it on the car.

```
echo "export ROS_DOMAIN_ID=6" >> ~/.bashrc  # The 6 here is the ROS_DOMAIN_ID. It does not
have to be 6; any value that conforms to the ROS_DOMAIN_ID rules will do.
source ~/.bashrc
ros2 run demo_nodes_py talker
```

2. At the same time, execute on the slave side [virtual machine]:

```
echo "export ROS_DOMAIN_ID=6" >> ~/.bashrc  # Here the value is kept consistent with the value on the host side
source ~/.bashrc
ros2 run demo_nodes_py listener
```

If the following is displayed: the topic published by the host can be subscribed to in time by the slave, which means grouped multi-machine communication has been achieved



2.3. Case 2

Control the movement of the little turtle through distributed communication

- The command run on host A

```
ros2 run turtlesim turtlesim_node
```

- The command run on host B

```
ros2 run turtlesim turtle_teleop_key
```

3. Notes

When setting the value of ROS_DOMAIN_ID, it is not arbitrary; there are also certain constraints:

1. It is recommended that the value of ROS_DOMAIN_ID be in the range [0, 101], including 0 and 101;
2. The total number of nodes within each domain ID is limited and must be less than or equal to 120;
3. If the domain ID is 101, the total number of nodes in that domain must be less than or equal to 54.

4. DDS Domain ID Value Calculation Rules (Advanced Knowledge)

The relevant calculation rules for the domain ID value are as follows:

1. DDS is based on the TCP/IP or UDP/IP network communication protocol. A port number must be specified during network communication, and the port number is represented by a 2-byte unsigned integer, whose value range is [0, 65535];
2. The assignment of port numbers also follows rules and cannot be used arbitrarily. According to the DDS protocol, 7400 is used as the starting port, so the available ports are [7400, 65535]. It is also known that according to the DDS protocol, each domain ID occupies 250 ports by default, so the number of domain IDs is: (65535-7400)/250 = 232, and the corresponding value range is [0, 231];
3. The operating system also reserves some ports. When using ports in DDS, these reserved ports must be avoided to prevent conflicts during use. Different operating systems reserve different ports. The final result is that, under Linux, the available domain IDs are [0, 101] and [215-231], while under Windows and Mac the available domain IDs are [0, 166]. In summary, for compatibility with multiple platforms, it is recommended that the domain ID be chosen within the range [0, 101].
4. Each domain ID occupies 250 ports by default, and each ROS2 node needs to occupy two ports. In addition, according to the DDS protocol, within the port segment of each domain ID, ports 1 and 2 are the Discovery Multicast port and the User Multicast port, and ports 11 and 12 are the Discovery Unicast port and the User Unicast port of the first node in the domain, while the ports occupied by subsequent nodes follow in sequence. Therefore, the maximum number of nodes in a domain ID is: (250-10)/2 = 120;
5. Special case: when the domain ID value is 101, the ports in the second half belong to the reserved ports of the operating system, and the maximum number of nodes is 54.

The above calculation rules only need to be understood.